

DTS: Delta-Based Tile Scoring for Accelerating Long-Context Attention

Anonymous Authors

Affiliation

Organization

City, Country

email@domain.com

Abstract—Long-context inference in large language models is constrained by the quadratic computational complexity of self-attention. To address this, we leverage the high sequential similarity of adjacent query and key vectors to propose Delta-based tile scoring (DTS). DTS uses content-adaptive subsets of these vectors to efficiently estimate tile importance for block-sparse attention. Experiments on LLaMA-3.1-8B-Instruct demonstrate that DTS closely matches full attention accuracy on RULER and LongBench while achieving a 3.3× end-to-end prefill speedup at a 512K context length. Furthermore, DTS generalizes across architectures and modalities. Importantly, it achieves these results while maintaining a substantially lower scoring cost than XAttention.

Index Terms—sparse attention, LLM inference, long context, hardware acceleration

I. INTRODUCTION

Long-context inference has become a critical and impactful application for Large Language Models (LLMs) [1]–[5]. For instance, coding agents [6] ingest entire codebases to reason about them, while other systems analyze extensive documents such as legal contracts, research papers, and financial reports. Furthermore, Retrieval-Augmented Generation (RAG) [7] integrates retrieved documents directly into the model context, grounding LLM responses in external knowledge. However, all these applications share a common bottleneck.

LLM inference consists of two stages: the prefill stage, which processes the entire input prompt in a single forward pass, and the decoding stage, which generates output tokens one by one. During the prefill stage, computing self-attention performs a computation that is quadratic in complexity, as each token must communicate with every other token.

As context length grows, the quadratic complexity of attention makes prefill increasingly expensive. Memory was the original wall; FlashAttention [8]–[10] and PagedAttention [11] removed it but did not address the time complexity: computing attention on long contexts is slow.

A large body of work aims to accelerate attention for long context tasks by exploiting the empirical sparsity of the attention matrix: most tokens only attend strongly to a small local window and a few distant positions, so many token pair interactions do not need to be computed at all. Early work used fixed sparse patterns [12]–[14]; more recent methods determine the pattern dynamically based on the input [15]–[17].

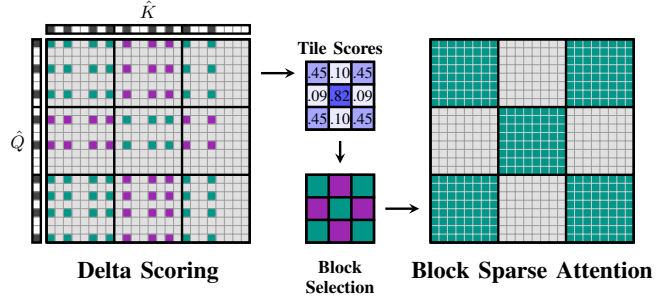


Fig. 1: DTS four-stage process: Compress queries and keys into $\hat{Q}$ and $\hat{K}$ and multiply them to estimate sparse attention scores during delta scoring. These individual scores are then summed to yield a single importance score for each tile. Finally, the highest-scoring tiles are selected to undergo block sparse attention.

This paper exploits the insight that adjacent tokens have similar key vectors, with high cosine similarity between neighboring keys. This redundancy in K is already documented in prior work [18], [19]. We show that the same holds for query vectors: adjacent queries are also highly similar.

We apply this insight as a tile importance estimator for block-sparse attention, choosing which tiles to compute while sampling far less of each tile than existing estimators.

II. RELATED WORK

The dominant strategy in attention acceleration is exploiting the fact that attention matrices are empirically sparse: most tokens attend strongly to only a small fraction of others. Child et al. [12], along with subsequent works like Longformer [13] and BigBird [14], were among the first to demonstrate this, observing that trained Transformer attention matrices concentrate on local windows and a few distant positions. They proposed fixed sparse patterns (local windows combined with strided or fixed summary positions) to approximate full attention with fewer FLOPs. The limitation is that a fixed pattern cannot adapt to the content of a given input. This motivated a shift to dynamic sparse attention, where the sparsity pattern is determined on the fly. These methods generally fall into two

categories: attention shape methods and block-sparse attention methods.

Block-sparse attention methods dynamically skip attention tiles that are estimated to be unimportant. SpargeAttn [20] selects a representative per block using mean pooling and scores tiles based on the block representatives. XAttention [17] uses the sum of antidiagonal values within each attention block as a lightweight proxy for tile importance. Both methods then compute attention only for the tiles scored as important.

Attention shape methods, in contrast, identify and exploit the specific attention pattern a head exhibits. For example, MInference [15]—a foundational and widely deployed approach—observes that LLM attention heads follow consistent structural patterns (A-shape, Vertical-Slash, and Block-Sparse) across inputs. It assigns each head its dominant pattern offline and builds sparse indices accordingly at inference time. Crucially, Block-Sparse heads lack a fixed, exploitable shape; for these, MInference falls back to block-sparse attention. Similar to SpargeAttn, it mean-pools each query and key block into a single representative and uses the resulting attention scores to select important tiles.

FlexPrefill [16] extends MInference by making it content-adaptive. Instead of assigning a shape to a head offline, FlexPrefill decides at inference time whether to treat a head as Vertical-Slash or block-sparse, based on the Jensen-Shannon divergence between the head’s attention and each candidate pattern. It also differs in how tiles are selected.

Our Delta-based tile scoring (DTS) is most closely related to XAttention [17] and SpargeAttn [20]. All three methods compute a cheap importance score to identify which tiles require exact computation. While XAttention sums the antidiagonal values of each block and SpargeAttn mean-pools blocks into single representatives, DTS compresses Q and K to perform a lightweight attention that captures tile importance. Furthermore, unlike XAttention’s reliance on rigid strides, DTS’s tile scoring remains fully content-adaptive.

III. OBSERVATIONS

High cosine similarity between adjacent key states has been previously observed [18].

In this section, we extend this analysis to both key and query states. We feed LLaMA-3.1 8B Instruct a sequence of 512 tokens sampled from WikiText-103 and record the states at layers 0, 8, 16, and 23. At each layer, we compute the average cosine similarity between all adjacent token pairs, averaged over heads. We compare two configurations: (1) **Natural Order (post-RoPE)**: states computed with the original token sequence, and (2) **Shuffled Order**: states computed after randomly shuffling the token positions, breaking positional adjacency.

Comparing the shuffled and natural orders allows us to isolate the source of the similarity. As shown in Figure 2, both key and query states exhibit high adjacent cosine similarity in their natural order. When the token order is shuffled, the similarity drops significantly for both keys and queries. This confirms that the high adjacent similarity is not solely an

artifact of the projection matrices or positional encodings, but is strongly driven by the inherent semantic and structural redundancy of adjacent tokens in natural text. Crucially, query states exhibit the same high local redundancy as key states, motivating a compression strategy that exploits both.

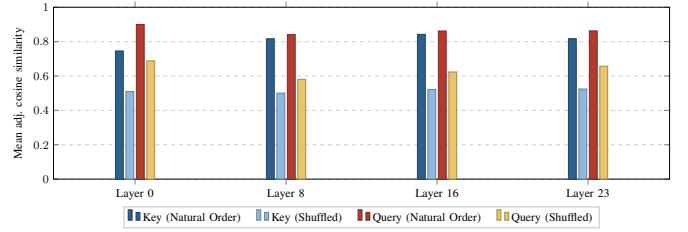


Fig. 2: Average cosine similarity of adjacent key and query states for LLaMA-3.1 8B Instruct, showing both natural token order (post-RoPE) and randomly shuffled token order.

IV. METHODOLOGY

In this section, we describe the core components of our proposed approach: Delta Packing and its extension to Delta-Based Tile Scoring (DTS).

A. Delta Packing

The foundational operation in our methodology is Delta Packing. Adjacent query and key vectors are highly similar at the token level, which means they can be compressed along the sequence dimension.

Algorithm 1 outlines the sequential Delta Packing procedure.

Algorithm 1: Sequential Delta Packing

Input: Matrix $\mathbf{K} \in \mathbb{R}^{N \times D}$, similarity threshold τ

Output: Packed matrix $\hat{\mathbf{K}}$, binary mask $\Delta \in \{0, 1\}^N$

1. $\Delta \leftarrow \mathbf{0}$, $\Delta[0] \leftarrow 1$
 2. $\mathbf{k}_{\text{anchor}} \leftarrow \mathbf{K}[0]$
 3. **for** $i = 1$ **to** $N - 1$ **do**
 4. **if** $\cos(\mathbf{K}[i], \mathbf{k}_{\text{anchor}}) < \tau$ **then**
 5. $\Delta[i] \leftarrow 1$
 6. $\mathbf{k}_{\text{anchor}} \leftarrow \mathbf{K}[i]$
 7. **end if**
 8. **end for**
 9. $\hat{\mathbf{K}} \leftarrow \mathbf{K}[\Delta == 1]$
 10. **return** $\hat{\mathbf{K}}, \Delta$
-

Parallel Delta Packing. In practice, we apply sequential delta packing to fixed-size chunks (e.g., 128 tokens) independently by setting each chunk’s first vector as an anchor, which breaks the sequential dependency. Otherwise, sequential packing has a linear time complexity with respect to sequence length, which would result in severe overhead in extremely long contexts.

B. Delta-Based Tile Scoring (DTS)

Building upon the Delta Packing concept, we introduce Delta-Based Tile Scoring (DTS) for block-sparse attention.

The DTS methodology operates as follows:

- 1) **Delta Pack:** Apply Delta packing on $\mathbf{Q}$ and $\mathbf{K}$ separately.
- 2) **Multiply:** Multiply the packed $\hat{\mathbf{Q}}$ and $\hat{\mathbf{K}}$ to get representative attention scores in tiles.
- 3) **Sum:** Sum the tile attention scores to get an importance score for each tile.
- 4) **Normalize:** Apply softmax to the tile scores across query rows to obtain tile weights.
- 5) **Threshold:** Select the highest-scoring tiles per row until their accumulated weights exceed a threshold $\tau = 0.9$.
- 6) **Compute:** Process the selected tiles using block-sparse attention and skip the remaining tiles.

Figure 3 provides a detailed algorithmic view of how $\mathbf{Q}$ and $\mathbf{K}$ are packed into $\hat{\mathbf{Q}}$ and $\hat{\mathbf{K}}$, and how these packed matrices are multiplied to obtain the sparse attention scores.

Figure 1 illustrates the complete end-to-end flow of this four-stage methodology, showcasing how the sparse attention scores are summed to compute tile importance, and how these tile scores determine which tiles are processed and which are skipped.

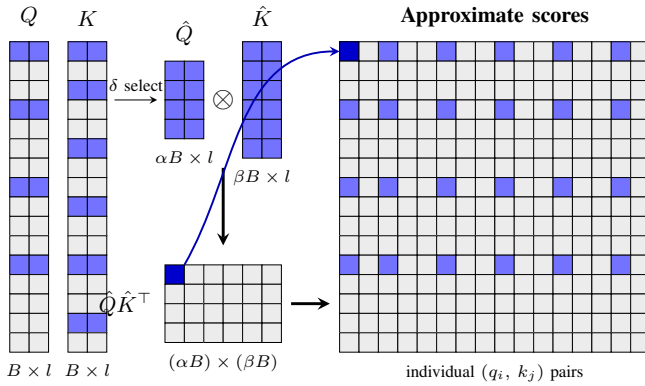


Fig. 3: Delta-based tile scoring. Delta pre-processing selects content-adaptive anchors (blue) from $\mathbf{Q}$ and $\mathbf{K}$ independently, producing $\hat{\mathbf{Q}}$ and $\hat{\mathbf{K}}$. Their product gives an $(\alpha B) \times (\beta B)$ score matrix; each entry (highlighted cell, arrow) corresponds to a single dot product at one position in the full $B \times B$ score tile.

C. Per-Head τ Calibration

A global threshold of $\tau = 0.9$ does not strictly guarantee retaining 90% of the true attention mass, because the tile scores are only estimates. To accurately achieve the target retention rate, similar to X-Attention, we perform an offline calibration step. During this step, the threshold τ is independently tuned for each attention head so that every head reliably retains its target mass. Detailed procedures for this calibration are presented in Appendix A.

V. EXPERIMENTAL SETUP

A. Benchmarks

We evaluate our method on two text benchmarks and one video benchmark. The text benchmarks are evaluated using LLaMA-3.1 [1] and Qwen2.5 [2], while the video benchmark uses Qwen2-VL [3].

1) Text Benchmarks:

a) **RULER:** RULER is a synthetic, retrieval-heavy benchmark that evaluates models across context lengths typically ranging from 4K to 128K tokens. The standard RULER setup includes 13 tasks with 100 samples per pass, making it computationally expensive. For development and ablations, we employ a lighter setup comprising 30 samples per task across 9 tasks.

b) **LongBench:** LongBench is a real-world benchmark featuring tasks such as coding, question answering, and summarization, with context lengths ranging from roughly 3K to 24K tokens. It covers 21 datasets across six task categories: single-document QA, multi-document QA, summarization, few-shot learning, synthetic retrieval, and code completion. We sample 100 instances per task and evaluate our main DTS results on 16 of these tasks.

2) Video Benchmarks:

a) **Video-MME:** Video-MME is a long-video understanding benchmark that tests multiple-choice question answering over videos, sampling each video at 1 frame per second. The benchmark comprises 2,700 videos with durations ranging from short (under 2 minutes) to long (30 to 60 minutes). Our main results evaluate the full 2,700-video test set, while our packing threshold ablations use a reduced 200-video subset.

B. Speedup Experiment

To measure raw attention speedup, we run an isolated prefill test independent of any downstream task, following the latency evaluation in MInference [15]. The model is fed a randomly sampled sequence from WikiText-103 across a range of context lengths. We measure the time to first token, which covers the full prefill stage. DTS is compared against PyTorch SDPA [21], [22], which dispatches to a FlashAttention-2 [9]. We perform 2 warmup and 10 measured runs.

VI. RESULTS

We evaluate delta-based tile scoring (DTS) against MInference, FlexPrefill, and XAttention. We exclude SparseAttention from our comparisons because its implementation is bundled with quantization. All baselines are run directly from their original GitHub repositories without reimplementing. We report DTS at two cosine thresholds, 0.72 and 0.78, both at a recall of $r=0.90$. We evaluate these methods on RULER (100 samples, 13 tasks) and LongBench (16 tasks), alongside an isolated speedup analysis from 4K to 512K tokens.

A. RULER

Table I reports RULER accuracy. DTS achieves near-lossless performance, with both threshold settings outperforming every sparse baseline tested. The performance gap is most

pronounced at 128K tokens: DTS maintains scores of 75.55 (cos 0.78) and 74.82 (cos 0.72), whereas XAttention drops to 70.96 ($s=16$) and 71.57 ($s=8$), and MInference falls more severely to 63.73.

TABLE I: RULER accuracy (%) comparing delta-based tile scoring against baseline and block-sparse methods on LLaMA-3.1 8B Instruct (100 samples, 13 tasks). DTS and FlexPrefill are both at recall/coverage target 0.90 ($r=0.90$ and $\gamma=0.90$ respectively). Best score per column in **bold**.

Method	4K	8K	16K	32K	64K	128K	Avg
Full attention	96.16	93.77	93.43	90.47	86.14	74.46	89.07
MInference	96.04	93.95	92.60	88.43	84.66	63.73	86.57
FlexPrefill	95.37	92.29	91.23	88.02	83.56	74.23	87.45
XAttn $s=16$	96.17	93.97	93.43	90.56	83.04	70.96	88.02
XAttn $s=8$	95.57	93.92	92.71	90.60	81.83	71.57	87.70
DTS cos 0.72	95.83	94.22	93.36	90.42	85.19	74.82	88.97
DTS cos 0.78	96.14	93.70	93.44	90.35	84.93	75.55	89.02

B. LongBench

Table II shows that DTS is the strongest sparse method on average, nearly matching full attention. However, performance varies by task. On multi-hop reasoning (2Wiki-MultiHopQA) and LCC, cos 0.72 matches or exceeds the baseline while cos 0.78 slightly trails. Summarization is robust across all methods, while DTS excels on passage retrieval and RepoBench-P, exactly matching or significantly beating full attention. While DTS slightly trails baselines on TREC for LLaMA, it achieves the highest score on Qwen, indicating this is not a systematic flaw.

C. Speedup and discussion

Figure 4 shows the end-to-end speedup for every method from 4K to 512K tokens. All sparse methods initially run slower than SDPA at short contexts due to scoring overhead, but accelerate as attention dominates the forward pass. MInference is the slowest to catch up, crossing break-even at 48K and trailing others until 100K tokens. FlexPrefill is the fastest up to 256K, but MInference overtakes it at 512K ($9.33\times$ vs. $4.83\times$). Meanwhile, DTS and XAttention track each other closely, crossing break-even near 16K and reaching roughly $3.3\times$ at 512K. At the 128K operating point, both DTS settings match or slightly exceed the end-to-end speed of XAttention ($2.02\times$ – $2.08\times$ for DTS vs. $1.89\times$ – $2.00\times$ for XAttention). This is driven by DTS’s significantly lower scoring overhead. For example, DTS cos 0.72 spends only 2.3% of the full-attention tile cost on scoring (1.59s), whereas XAttention $s=16$ spends 6.2% (2.57s) at a similar attention density (17.6% vs 17.1%).

D. Cosine threshold ablation

The cosine threshold τ controls how aggressively DTS packs the delta representation used for tile scoring: a lower τ keeps fewer rows, making the score cheaper but less accurate, while a higher τ keeps more rows for a more accurate but more expensive score. We sweep $\tau \in \{0.70, 0.72, 0.75, 0.78, 0.80\}$ at fixed recall $r=0.90$ and report speedup, RULER, and

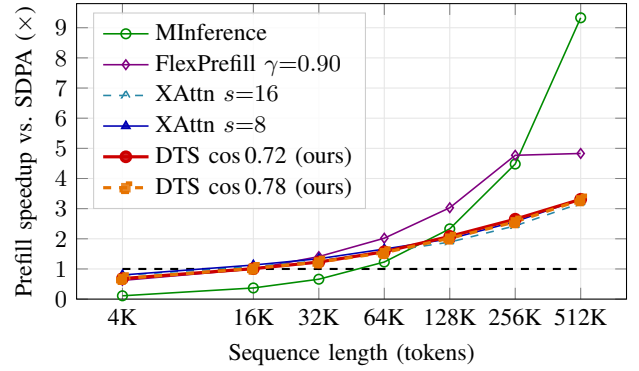


Fig. 4: End-to-end prefill speedup relative to SDPA across sequence lengths on LLaMA-3.1 8B Instruct ($4\times H100$, layer-sharded). Points above the dashed break-even line ($1.0\times$) are faster than full attention.

LongBench. Because this sweep repeats the evaluation across five configurations, we use the reduced RULER (30 samples, 9 tasks) and reduced LongBench (9 tasks).

The sweep trades scoring cost against tile count, and the two roughly cancel out (Table III). Lower thresholds score faster (1.6% of full tile cost, 1,447 ms at cos 0.70) but need more tiles to reach 90% attention mass, since the ranking is coarser (19.0% of tiles, 3,094 ms of attention). Higher thresholds give a sharper ranking that hits the same recall with fewer tiles (13.3% of tiles, 2,140 ms of attention at cos 0.80) but cost more to score (8.7%, 2,885 ms). Because scoring and attention move in opposite directions, the end-to-end speedup stays roughly flat: it peaks at cos 0.72 ($2.10\times$), stays close at cos 0.70 ($2.08\times$), and only drops off at the high end ($2.03\times$ at cos 0.78, $1.97\times$ at cos 0.80).

RULER is essentially lossless across the whole sweep (Table III): every setting lands within 0.4 points of the others.

LongBench is more sensitive to τ (Table III). Accuracy peaks at cos 0.72 (39.21) and falls off at both ends. cos 0.72 is the best overall choice: fastest and best on LongBench. cos 0.78 is the accuracy-maximizing choice, holding the best RULER score (91.80) at a still-good $2.03\times$. We carry both forward as the two settings compared against the other methods.

E. Cross-Model Generalization: Qwen2.5-7B

The results above use LLaMA-3.1 8B Instruct throughout. To check whether DTS’s behavior is specific to that model, we repeat the full RULER, LongBench, and speedup evaluation on Qwen2.5-7B-Instruct-1M, using the same two operating points (cos 0.72 and cos 0.78 at $r=0.90$) and the same baselines.

a) *RULER.*: Table IV reports RULER accuracy on Qwen. MInference (90.35) and both XAttention strides (90.25 at $s=16$, 90.23 at $s=8$) tie full attention (90.33); DTS trails by 1.4–2.2 points (88.89 at cos 0.78 and 88.12 at cos 0.72). FlexPrefill collapses to 53.50.

To trace the source of this performance gap, we analyze the 13 individual RULER tasks at 128K. While DTS matches

TABLE II: LongBench scores on LLaMA-3.1 8B Instruct (100 samples per task, 16 tasks). DTS and FlexPrefill are both at recall/coverage target 0.90 ($r=0.90$ and $\gamma=0.90$ respectively). NQA = NarrativeQA, Qas = Qasper, MFQ = MultifieldQA-en, HotP = HotpotQA, 2Wiki = 2WikiMultiHopQA, Mus = MuSiQue, GovR = GovReport, QMS = QMSum, MNews = MultiNews, TREC = TREC, TQA = TriviaQA, SAM = SAMSum, PCnt = PassageCount, PassR = PassageRetrieval-en, LCC = LCC, RepoB = RepoBench-P. Best score per column in **bold**.

Method	NQA	Qas	MFQ	HotP	2Wiki	Mus	GovR	QMS	MNews	TREC	TQA	SAM	PCnt	PassR	LCC	RepoB	Avg
Full attention	31.91	43.06	56.25	54.87	47.97	34.88	34.47	25.27	27.15	67.00	89.47	44.56	7.58	99.00	51.94	45.12	47.53
MInference	31.74	42.25	56.50	54.70	50.17	31.80	34.51	25.27	27.13	67.00	89.69	44.09	5.00	94.00	52.14	47.32	47.08
FlexPrefill	26.02	39.00	55.19	55.11	38.53	28.73	33.44	25.53	26.93	67.00	89.29	44.58	2.00	83.00	54.02	48.07	44.78
XAttn $s=16$	31.01	37.56	56.65	53.35	48.37	32.71	34.83	24.37	26.46	67.00	89.80	45.42	5.33	92.00	53.94	44.74	46.47
XAttn $s=8$	29.84	41.08	54.43	53.23	47.59	31.01	34.49	26.37	26.46	67.00	89.77	44.53	4.00	93.00	52.67	48.61	46.50
DTS cos 0.72	31.62	42.86	57.04	53.29	47.88	32.87	34.67	25.66	26.97	64.00	89.30	44.74	6.50	99.00	54.32	49.48	47.51
DTS cos 0.78	31.81	43.60	57.19	54.51	45.27	32.72	35.26	25.35	26.67	66.00	89.30	43.83	6.83	99.00	50.62	49.26	47.33

TABLE III: End-to-end prefill speedup (128K tokens) and accuracy (%) on RULER and LongBench for DTS cosine thresholds on LLaMA-3.1 8B Instruct ($r=0.90$). Attention density is the fraction of computed sparse attention. Score% represents the scoring overhead. Score ms and Attn ms indicate the respective phase latencies.

Method	Speedup	Density	Score%	Score (ms)	Attn (ms)	RULER	LongBench
Full attention	1.00×	100%	—	—	—	91.52	39.54
cos 0.70	2.08×	19.0%	1.6%	1,447	3,094	91.69	38.95
cos 0.72	2.10×	17.6%	2.3%	1,611	2,853	91.51	39.21
cos 0.75	2.09×	15.7%	3.8%	1,954	2,545	91.42	39.08
cos 0.78	2.03×	14.2%	6.3%	2,445	2,284	91.80	39.15
cos 0.80	1.97×	13.3%	8.7%	2,885	2,140	91.58	38.61

TABLE IV: RULER accuracy (%) comparing delta-based tile scoring against baseline and block-sparse methods on Qwen2.5-7B-Instruct-1M (100 samples, 13 tasks). DTS and FlexPrefill are both at recall/coverage target 0.90 ($r=0.90$ and $\gamma=0.90$ respectively). Best score per column in **bold**.

Method	4K	8K	16K	32K	64K	128K	Avg
Full attention	94.10	93.67	93.54	91.17	88.07	81.41	90.33
MInference	94.16	94.00	94.03	90.99	87.68	81.23	90.35
FlexPrefill	74.39	61.74	55.66	59.71	32.38	37.10	53.50
XAttn $s=16$	94.02	93.55	93.66	90.85	88.21	81.22	90.25
XAttn $s=8$	94.06	93.53	93.60	91.02	87.96	81.23	90.23
DTS cos 0.72	94.01	92.38	92.56	87.45	83.69	78.60	88.12
DTS cos 0.78	94.02	93.54	92.59	89.38	84.54	79.26	88.89

or beats XAttention on 11 tasks, it falls short on variable tracking (vt) and common word extraction (cwe). Both are aggregation tasks that require combining information from multiple positions rather than retrieving a single needle. DTS’s performance drops sharply on these (e.g., 53.8 and 13.8 at cos 0.72, compared to full attention’s 75.8 and 27.3), indicating a trade-off where DTS remains strong on retrieval but struggles with broad aggregation.

b) LongBench.: Table V shows LongBench scores on Qwen. DTS ties full attention (46.76): 46.19 at cos 0.72 and 46.20 at cos 0.78, tracking both XAttention variants (46.46 at $s=8$, 46.66 at $s=16$).

DTS only underperforms full attention on MuSiQue (29.22–29.88 vs. 32.70) and 2WikiMultiHopQA (52.46–53.31 vs. 55.45). Because both are multi-hop tasks that require syn-

thesizing evidence across passages, this result reinforces the aggregation limitation observed in RULER. Additionally, MInference’s average is marked with a † because it failed to complete QMSum and RepoBench-P at 128K.

c) Speedup.: Figure 5 illustrates end-to-end speedups on Qwen up to 512K (DTS and XAttention are capped at 256K due to OOM errors). The trends largely reflect those seen on LLaMA. MInference is the slowest up to 64K but overtakes FlexPrefill at 512K, reaching $4.81\times$ (a smaller gain than on LLaMA). FlexPrefill is the fastest up to 256K before dipping to $3.87\times$ at 512K. DTS and XAttention perform similarly, though DTS maintains a clear speed advantage at 128K and 256K ($1.57\times$ and $1.81\times$ vs. XAttention’s $1.45\times$ and $1.74\times$).

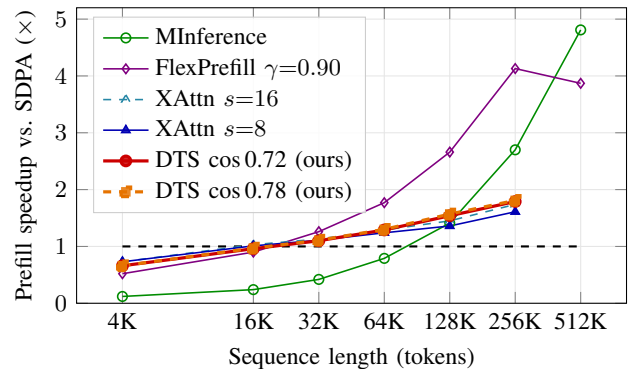


Fig. 5: End-to-end prefill speedup relative to SDPA across sequence lengths on Qwen2.5-7B-Instruct-1M ($4\times H100$, layer-sharded). XAttention and DTS are reported to 256K; both OOM at 512K on this hardware.

F. Video Understanding: Qwen2-VL

We evaluate DTS on Video-MME (1 fps, no subtitles) using Qwen2-VL-7B-Instruct. We empirically observed that packing by Euclidean distance yields better performance on video tokens than cosine similarity, so we report DTS using Euclidean packing (see Appendix B for this ablation study). We measure accuracy on the full test set (2,700 videos) and prefill speedup (attention latency only) on a 10-video timing sample per duration split.

TABLE V: LongBench scores on Qwen2.5-7B-Instruct-1M (100 samples per task, 16 tasks). DTS and FlexPrefill are both at recall/coverage target 0.90 ($r=0.90$ and $\gamma=0.90$ respectively). Task abbreviations as in Table II. Best score per column in **bold**. †MInference failed on 2 of 16 tasks (QMSum, RepoBench-P); its reported average is over the 14 completed tasks and is not directly comparable to the other rows.

Method	NQA	Qas	MFQ	HotP	2Wiki	Mus	GovR	QMS	MNews	TREC	TQA	SAM	PCnt	PassR	LCC	RepoB	Avg
Full attention	29.26	45.12	51.49	58.34	55.45	32.70	35.22	23.99	25.69	71.00	83.31	45.24	10.00	100.00	48.40	32.92	46.76
MInference†	29.38	45.36	51.85	56.77	54.65	33.47	35.20	–	25.98	71.00	83.31	46.32	10.00	99.00	47.63	–	(44.4)†
FlexPrefill	24.68	39.46	50.63	48.72	50.58	21.06	33.61	24.67	25.42	70.00	78.64	46.11	2.00	59.00	45.18	33.61	40.84
XAttn $s=16$	29.84	44.74	51.41	57.63	55.62	32.60	34.84	24.00	26.09	70.00	83.18	46.00	12.00	100.00	46.70	31.97	46.66
XAttn $s=8$	30.26	45.77	52.31	58.16	53.78	31.62	35.18	24.09	26.00	71.00	83.51	45.83	9.00	100.00	45.14	31.70	46.46
DTS $\cos 0.72$	29.89	46.48	52.04	58.56	53.31	29.22	35.12	24.19	25.86	72.00	83.17	46.51	4.00	98.00	45.57	35.15	46.19
DTS $\cos 0.78$	28.16	42.79	50.85	55.59	52.46	29.88	35.11	24.33	25.57	74.00	83.84	46.18	10.00	97.00	46.52	36.90	46.20

Table VI reports results against the same baselines used in text experiments. DTS at $\delta=15$ is our primary configuration. It closely tracks full attention accuracy (60.8 vs. 62.1 overall) while providing the best balance of speed and accuracy on the critical long-video split: it scores 52.6 and runs at $1.33\times$ full attention, outperforming XAttention (51.7, $1.18\times$). MInference retains the highest accuracy (62.4) but remains slower than full attention across all buckets. FlexPrefill provides the highest speedup (2.18 $\times$) but trails DTS in long-video accuracy (51.6).

TABLE VI: Video-MME accuracy and prefill speedup on Qwen2-VL-7B-Instruct (Video-MME @ 1fps, no subtitles). Accuracy (%) is on the full test set ($n=2700$). speedup ($\times$ full) is attention-kernel latency (self_attn_ms) relative to full attention on a 10-videos-per-bucket timing sample. Best accuracy per column in **bold**.

Method	Accuracy (%)				Speedup ($\times$ full)		
	Short	Med	Long	Avg	Short	Med	Long
Full attention	72.3	61.0	53.0	62.1	1.00	1.00	1.00
MInference	72.8	61.2	53.3	62.4	0.42	0.62	0.61
FlexPrefill	72.1	61.3	51.6	61.7	1.41	2.12	2.18
XAttn $s=16$ ($r=0.90$)	72.0	59.9	51.7	61.2	0.95	1.22	1.18
DTS $\delta=15$ ($r=0.90$)	71.1	58.6	52.6	60.8	0.98	1.21	1.33

VII. CONCLUSION

Among the baselines we tested, there is no silver bullet. Each method trades speed against accuracy and generalization in a different way. The shape-oriented methods (MInference and FlexPrefill) excel in speedup, yet FlexPrefill struggles with model generalization and MInference realizes its speedup late.

MInference reaches exceptional speedups at very long contexts (256K to 512K), but it rises late and is slower than the baseline up to 128K on Qwen2.5. This shows up on video, where the longest clips sit around 32K tokens and MInference is much slower than full attention ($0.42\times$ to $0.62\times$). It holds up accuracy and generalizes to models well. However, its RULER accuracy degrades at 128K on LLaMA (63.73 versus 74.46 for full attention). Overall, its solid performance proves its place in production.

FlexPrefill has a smoother speedup curve. It delivers speedups at shorter contexts even though it cannot reach the

dramatic 512K speedup of MInference. Although it is competitive on speed, it suffers accuracy degradation on LLaMA RULER’s 128K (74.23 versus 74.46) and on LongBench average (44.78 versus 47.53). More importantly, its accuracy breaks on every Qwen text benchmark (RULER 53.50 versus 90.33), which points to a cross-model generalization failure.

The block-sparse methods (XAttention and DTS) retain accuracy better across the benchmarks, yet they cannot reach the impressive speedups of the shape-oriented methods.

XAttention generally has solid accuracy retention and cleanly generalizes to different models and modalities. On the Qwen RULER benchmark it scores considerably higher than DTS (90.25 versus 88.12–88.89), but it falls back on LLaMA compared to DTS (88.02 versus 89.02 on RULER). Its overall accuracy is tied with DTS on video, yet on the long bucket DTS has an advantage (52.6 versus 51.7) while providing higher speedup ($1.33\times$ versus $1.18\times$).

Overall DTS generally excels at accuracy retention and shows solid model and modality generalization. The one exception is the Qwen RULER aggregation tasks (variable tracking and common word extraction), where its accuracy degrades, though this does not carry over to the real-world LongBench benchmark. It is the best method for accuracy retention on LLaMA and in long video understanding. Compared to XAttention, at all configurations and sequence lengths, it either matches or beats XAttention’s speedup due to its much lighter scoring phase. The only reason we cannot say it dominates XAttention is the Qwen RULER aggregation degradation.

A. Future Work

Several directions could push DTS further. First, thresholding can be refined by applying separate limits to Q and K , or using more aggressive thresholds in later layers. Second, DTS currently lacks a tile normalization scheme, meaning that some tiles inherently score higher simply because they contain more unpruned vectors. Finally, future work will evaluate DTS on larger models, test it on the InfiniBench benchmark, and apply it to image and video generation tasks.

REFERENCES

- [1] A. Dubey *et al.*, “The llama 3 herd of models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [2] Q. Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [3] P. Wang *et al.*, “Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution,” *arXiv preprint arXiv:2409.12191*, 2024.
- [4] M. Reid *et al.*, “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context,” *arXiv preprint arXiv:2403.05530*, 2024.
- [5] H. Liu, M. Zaharia, and P. Abbeel, “Ring attention with blockwise transformers for near-infinite context,” *arXiv preprint arXiv:2310.01889*, 2023.
- [6] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [7] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [8] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16 344–16 359.
- [9] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations*, 2024.
- [10] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, “Flashattention-3: Fast and accurate attention with asynchrony and low-precision,” *arXiv preprint arXiv:2407.08608*, 2024.
- [11] W. Kwon *et al.*, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [12] R. Child, S. Gray, A. Radford, and I. Sutskever, “Generating long sequences with sparse transformers,” *arXiv preprint arXiv:1904.10509*, 2019.
- [13] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv:2004.05150*, 2020.
- [14] M. Zaheer *et al.*, “Big bird: Transformers for longer sequences,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [15] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention,” in *Advances in Neural Information Processing Systems*, 2024.
- [16] X. Lai, J. Lu, Y. Luo, Y. Ma, and X. Zhou, “FlexPrefill: A context-aware sparse attention mechanism for efficient long-sequence inference,” in *International Conference on Learning Representations*, 2025.
- [17] R. Xu, G. Xiao, H. Huang, J. Guo, and S. Han, “XAttention: Block sparse attention with antidiagonal scoring,” *arXiv preprint arXiv:2503.16428*, 2025.
- [18] Z. Wang, B. Jin, Z. Yu, and M. Zhang, “Model tells you where to merge: Adaptive KV cache merging for LLMs on long-context tasks,” *arXiv preprint arXiv:2407.08454*, 2024.
- [19] J. Qi, C. Gao, Z. Ren, and Q. Chen, “DeltaLLM: A training-free framework exploiting temporal sparsity for efficient edge LLM inference,” *TODO: add arXiv ID or venue*, 2024.
- [20] J. Zhang, C. Xiang, H. Huang, J. Wei, H. Xi, J. Zhu, and J. Chen, “SpargAttention: Accurate and training-free sparse attention accelerating any model inference,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025.
- [21] A. Paszke *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in neural information processing systems*, vol. 32, 2019.
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

APPENDIX A

PER-HEAD τ CALIBRATION DETAILS

To achieve the exact target retention of attention mass despite the tile scores being only estimates, we employ an

offline calibration procedure for the threshold τ . Specifically, we process a calibration set through the network and compute both the estimated tile scores from $\hat{\mathbf{Q}}$ and $\hat{\mathbf{K}}$ as well as the exact attention mass. For each head, we adjust its threshold τ_h until the actual retained attention mass matches the desired target (e.g., 90%). These per-head thresholds are then fixed and used during inference, ensuring reliable and uniform sparsity scaling across layers.

APPENDIX B

VIDEO-MME ABLATIONS

Table VII sweeps packing metrics on a 200-video subset. Euclidean distance consistently outperforms cosine similarity on long videos. Our chosen operating point, $\delta=15$, achieves the highest overall (60.5) and long-video (50.0) scores. While its scoring ratio (9.7%) is higher than XAttention’s (6.25%), it retains significantly fewer tiles (30.5% vs. 40.7%), making it faster end-to-end. A more aggressive threshold, $\delta=19$, drops the scoring ratio to 1.4% and retains only 23.6% of tiles, offering a promising high-speed alternative at the cost of some accuracy.

TABLE VII: Packing metric ablation on Video-MME (200-video subset), Qwen2-VL-7B-Instruct, recall $r=0.90$, count-weighting off throughout.

Metric	τ/δ	Overall	Short	Medium	Long	Retained tiles	Scoring ratio
XAttn $s=16$	–	59.0	75.0	59.7	44.6	40.7%	6.25%
Cosine	$\tau=0.90$	59.5	78.1	54.8	47.3	33.6%	45.8%
Cosine	$\tau=0.80$	58.5	76.6	56.5	44.6	32.4%	14.2%
Cosine	$\tau=0.75$	56.5	79.7	48.4	43.2	30.6%	7.4%
Cosine	$\tau=0.72$	58.5	81.2	50.0	45.9	29.3%	5.0%
Euclidean	$\delta=13$	59.5	78.1	56.5	45.9	32.8%	21.2%
Euclidean	$\delta=15$	60.5	76.6	56.5	50.0	30.5%	9.7%
Euclidean	$\delta=17$	58.5	76.6	53.2	47.3	27.1%	3.8%
Euclidean	$\delta=19$	57.5	75.0	48.4	50.0	23.6%	1.4%